

# Network-Attached FPGA Inference Scanning: A Bit-Exact SAN Catastrophe Scan over 100G Ethernet, and a Cautionary Tale About ARP

Demetrios Chiuratto Agourakis  
ORCID: 0009-0001-8671-8878

2026-08-13

## Abstract

We port a previously-accepted, production FPGA kernel — a catastrophe-exit scan for early-exit inference architectures (SAN-ResNet-50, SAN-ViT-large), validated bit-exact over PCIe DMA at 511 Msamples/s on an AMD Alveo U250 (spec T3, verdict T3\_GREEN) — to receive its input directly from a 100G Ethernet fibre instead of host DMA. The kernel’s arithmetic is unchanged; only the data-delivery interface changes, from an AXI4 memory-mapped read to an AXI4-Stream fed by a third-party UDP/IP network stack (Xilinx/AMD’s VNx). We report two results. First, a correctness result: the network-attached kernel reproduces the DMA-attached kernel’s output bit-for-bit — histogram, catastrophe count, and accumulated MAC cost — across cohorts up to 4,000,003 samples, confirmed in a batch of ten consecutive, independent hardware runs with zero data loss. Second, a systems-debugging result: an initial throughput measurement attempt produced what appeared to be hardware instability — a third-party network IP core that would intermittently stop accepting traffic regardless of the offered send rate, recoverable only by a full bitstream reload. We show this was not a hardware fault at all: it was ARP cache staleness in the host’s Linux IP stack, which silently discards or delays the first datagram of a burst sent after any idle period. A three-packet ICMP warm-up immediately before each data burst eliminates the failure entirely (10/10 clean runs post-fix, including a deliberately adversarial run with the ARP entry forcibly deleted). We report this debugging narrative in full because the failure signature — a third-party accelerator that “just needs a reset” under unpredictable conditions — is a common and expensive misdiagnosis in FPGA/network co-design, and the actual root cause was one layer up the stack from where we first looked.

**Keywords:** FPGA, Alveo U250, 100G Ethernet, RoCE fabric, network-attached accelerators, early-exit inference, catastrophe scan, VNx, HLS, bit-exact validation, ARP, systems debugging

**Category:** Computer Science > Distributed, Parallel, and Cluster Computing / Hardware Architecture

**License:** CC BY 4.0

**Status:** preprint draft, 2026-08-13. Companion artifact repository: [fpga/san-net/](#) (kernel, host controller, injector, test harness, and a complete build/debug log preserved in [README.md](#)).

## 1 Introduction

Early-exit inference architectures (SAN-ResNet-50, SAN-ViT-large) commit to a prediction at the first internal stage whose confidence clears a threshold, avoiding the compute cost of deeper stages. The *catastrophe scan* kernel implements this decision as a priority encoder over quantized Q0.15 confidences, with MAC-cost metering that charges each sample the real, architecture-specific FLOP prefix of its exit point — a host-programmable lookup table.

This kernel was previously accepted onto an AMD Alveo U250 (spec T3), fed by host DMA over PCIe, and validated bit-exact against a software golden model at 511 Msamples/s. That result establishes the kernel’s correctness and its DMA-bound throughput ceiling. It leaves open a different question: what if the confidences do not originate on the host at all, but arrive over the network from a producer elsewhere in a cluster — a GPU inference node, for instance, streaming logits to an FPGA scanner as a disaggregated pipeline stage? This motivates a **network-attached** variant: the same kernel, unchanged, fed by a 100G Ethernet link instead of DMA.

We build this variant on top of VNx (`xup_vitis_network_example`), AMD’s reference UDP/IP network stack for the U250, and report what we found in two parts: the correctness result (Section 4), and the throughput measurement attempt that initially looked like a hardware reliability problem and turned out to be a one-line host networking fix (Section 5). We include the latter at the same level of detail as the former because, in our reading of the FPGA-networking literature and vendor documentation, this specific failure mode — third-party network IP appearing to wedge under load, with the true cause sitting in the host’s neighbor-discovery cache rather than the fabric — is under-documented, and we believe the debugging method is as reusable as the result.

## 2 Background

**The accepted kernel (spec T3).** `krnl_san_scan` implements: (i) a catastrophe scan — for each sample, a comparator tree finds the first exit point whose confidence clears a threshold `q_delta`; samples that clear no threshold propagate to the final head and are counted as catastrophes; and (ii) FLOP metering — each sample is charged the exact MAC-prefix cost of its exit point from a host-loaded 64-bit lookup table, accumulated as a 64-bit total. Confidences arrive pre-quantized (Q0.15, no floating point on the card). The DMA-attached variant packs four samples per 512-bit AXI beat (`LANES=4`) and is throughput-bound by the PCIe/DMA path at 511 Msamples/s, not by kernel logic (`II=1`).

**The Alveo U250 and VNx.** The U250 is a Xilinx/AMD Virtex UltraScale+ VU13P card (4 SLRs, 1.72M LUTs, 12,288 DSPs) with dual QSFP28 cages, each capable of 100G Ethernet via a hardened CMAC block. VNx is AMD’s reference implementation of a UDP/IP network stack for this platform: a `cmac` kernel (the Ethernet MAC/PCS, including RS-FEC), a `networklayer` kernel (ARP responder, ICMP responder, and a fixed-size UDP socket table that routes matching (`theirIP`, `theirPort`, `myPort`) triples to per-socket AXI-Stream ports), and helper DMA kernels (`krnl_mm2s/krnl_s2mm`) for moving stream data to/from card DRAM. The `networklayer`’s UDP socket table supports up to `MAX_SOCKETS` entries (16 in our build), each entry occupying an 8-byte stride even though its constituent fields are 4 bytes — a detail that, if assumed rather than read from the generated kernel descriptor, silently misconfigures socket routing.

## 3 Architecture

### 3.1 Kernel changes

The kernel’s arithmetic core is untouched. Four changes port it to a streaming interface:

- (1) The input argument becomes `hls::stream<ap_axiu<512,0,0,16>` in place of a `bus_t*` pointer with an explicit index.
- (2) The per-beat read becomes `samples_in.read().data` in place of `samples[w]`.

- (3) `#pragma HLS INTERFACE axis port=samples_in` replaces the `m_axi` interface pragma for that argument.
- (4) The 16-bit `TDEST` field is retained to match VNx’s application-side stream convention, which uses `TDEST` to demultiplex per-socket traffic.

Sample packing is unchanged and remains the compatibility contract with the accepted artifact: one sample is 128 bits, holding up to seven 15-bit Q0.15 confidence fields, field  $k$  occupying bits  $[15k+14 : 15k]$ ; one 512-bit AXI beat carries four samples (`LANES=4`). We verified this packing two independent ways before any hardware run: a bit-level reimplement in plain `uint64_t` arithmetic checked against the `ap_uint`-based packer across 4,000 randomized cases (including the samples whose 15-bit fields straddle the 64-bit boundary, index 4 of 7), and a live packet capture on loopback, decoded field-by-field against an independently reimplemented linear congruential generator, with the high padding bits of each 128-bit record checked for zero.

### 3.2 Composition and placement

The `v++` link composes four kernel instances and constrains their connectivity:

```
fibre -> cmac_1 -> networklayer_1 -> krnl_san_scan_net_1 -> DDR -> host
                        ^
                        |
                krnl_mm2s_1  (closes the unused TX-from-application
                             path; v++ requires every declared
                             stream port connected, even one this
                             variant does not use for output)
```

We use the *second* CMAC/network-layer instance pair (`cmac_1`, `networklayer_1`), not the first: register-level inspection via the PCIe BAR showed the first QSFP cage on our card is uncabled. Discovering this by direct register read rather than assuming index 0, and subsequently addressing every core by name via `xrt::ip` rather than by an assumed base address, avoided a multi-hour misconfiguration during bring-up.

The compute kernel is constrained to SLR3 (`slr=krnl_san_scan_net_1:SLR3`), the least-occupied super-logic region after VNx’s own footprint (VNx occupies roughly 16% of SLR1 and 20% of SLR2; SLR3 sits under 1% before our kernel is added). Our kernel adds 11,299 LUTs (0.65% of the device), 6,195 FFs, and 8 BRAMs, at `II=1` and a 3-cycle iteration latency (`csynth: Fmax 342.47 MHz`).

### 3.3 Timing closure

At the requested 250 MHz kernel clock, place-and-route did **not** close timing: worst negative slack was  $-0.928$  ns on the `clk_kernel_00` domain (2,508 of 212,143 endpoints failing, all among a much larger sea of passing paths; the ten most-negative individual paths in the routed timing report all terminate in `krnl_mm2s_1`’s AXI-Lite control interface — a stub kernel we instantiate only to satisfy VNx’s stream connectivity requirement, not our compute path — though the timing report does not itemize enough endpoints to rule out other contributors). Vivado’s automatic frequency scaling reduced the kernel clock to 234.6 MHz, which is where the implemented design actually closes. This is not a throughput-limiting compromise for this application: at 512 bits per beat, 234.6 MHz sustains 120.1 Gbit/s of *kernel* bus bandwidth, above the 100 Gbit/s Ethernet line rate that ultimately bounds this system.

## 4 Correctness

### 4.1 Method

We built a two-process test harness. `ctl_san`, running on the host that owns the U250 (over XRT/PCIe, not over the network), programs the CMAC (enables RS-FEC, resets, and polls alignment — link-up was consistently observed within 0.5 s of reset in every run), writes the network layer’s identity (MAC/IP/gateway) and a single valid UDP socket-table entry, arms the kernel with a host-computed lookup table and the run parameters, and — critically — embeds the same golden-model computation the kernel is supposed to perform, so that every run self-verifies rather than relying on visual inspection of printed histograms. `inject_san`, running on a separate cluster node reachable over the same fabric, generates a cohort with a deterministic linear congruential generator and transmits it as UDP datagrams, one packet per `sendto` system call (Section 5 explains why this specific choice — one packet per syscall, no batching — is load-bearing).

### 4.2 Results

The kernel argument layout — read from the linked `.xclbin`’s embedded kernel descriptor via `xclbinutil -info`, not assumed — places the stream argument at index 0 and the scalar/buffer arguments at indices 1, 2, 3, 4, 5, 6, 7 for the cost LUT, `q_delta`, `n_points`, `n_samples`, histogram output, catastrophe count, and FLOP total respectively; the lookup table is 8 entries of 64-bit MAC-prefix costs and the histogram is 8 bins — both values taken from the kernel source rather than assumed, after an initial control-host implementation with a 32-bit, `n_conf`-sized LUT and a 256-bin histogram would have silently produced wrong numbers had it not been caught before the first hardware run.

Across the full campaign — an initial pair of runs at 100,003 and 4,000,003 samples, further runs during throughput characterization (Section 5), and a final confirmation batch of ten consecutive, independent 4,000,003-sample runs — every run in which the UDP payload was not intentionally overwhelmed (Section 5.1) reproduced the golden model’s histogram, catastrophe count, and FLOP total exactly. The confirmation batch is representative:

```
n_samples = 4,000,003 (x 10 independent runs)
catastrophes: 711,634 (all 10 runs)
FLOPs:      67,143,502,000,000 (all 10 runs)
histogram:   1,001,101 / 749,117 / 562,124 / 421,732 / 316,871 /
             237,424 / 711,634 / 0    (all 10 runs)
packet loss: 0 (network-layer frame counter delta across the 10 runs:
              7,148-7,170 against 7,143 data packets sent per run --
              consistent with a small amount of ordinary background
              broadcast/ARP traffic sharing the link, not data loss)
```

We regard this as sufficient evidence that the streaming interface change (Section 4) introduces no semantic divergence from the DMA-attached, previously-accepted kernel.

## 5 Throughput: an instability that was not where we thought it was

### 5.1 A genuine hardware limit, quickly found

Single-packet-per-syscall UDP transmission (`sendto`,  $\sim 104$  Msamples/s natural host rate) worked reliably in our first several hardware runs. To raise throughput, we batched transmission with `sendmmsg`, submitting 64 datagrams per system call. Send-side throughput rose

to 146.6 Msamples/s — but 97.7% of the resulting packets never reached the kernel (a network frame counter increased by only 167 of the 7,143 packets sent). This is a real and unsurprising finding: VNx’s `networklayer` has small on-chip FIFOs and no explicit flow-control back to the sender; a burst of 64 datagrams issued by the OS in immediate succession exceeds what those FIFOs can absorb, and excess frames are silently dropped. We do not consider this specific finding surprising or noteworthy on its own — it is the ordinary behavior of a UDP/IP stack sized for moderate, not bursty, offered load — but its *consequence* was: after this packet loss, the FPGA’s network path stopped accepting **any** further traffic, including from a reverted, previously-reliable single-packet sender, and including trivial 8-sample test cohorts. Reprogramming only the reconfigurable region (`xrt-smi program`) did not clear this state. Recovery required reloading the platform’s *base* bitstream via the card’s persistence service (which additionally re-runs CMAC bring-up) before reprogramming our variant — a several-minute round trip we needed six times over the course of this work.

## 5.2 The instability that was not the FIFO

Having removed `sendmmsg` entirely (reverting to one packet per `sendto` call, with an optional inter-call delay implemented as a token bucket to explicitly pace below the batching-induced failure mode above), we attempted a rate sweep to find a sustainable ceiling. The results did not describe a clean threshold:

| Attempt                                                            | Rate             | Outcome                              |
|--------------------------------------------------------------------|------------------|--------------------------------------|
| Unpaced <code>sendto</code> (natural host rate)                    | 12.6–13.4 Gbit/s | correct in each attempt at this rate |
| Unpaced <code>sendto</code> (natural rate, varies with scheduling) | 17.2 Gbit/s      | <b>failed</b>                        |
| Paced <code>sendto</code>                                          | 8.0 Gbit/s       | correct, 2 consecutive runs          |
| Paced <code>sendto</code>                                          | 12.0 Gbit/s      | correct, 1 run                       |
| Paced <code>sendto</code> , same rate as the first successful pair | 8.0 Gbit/s       | <b>failed</b>                        |

Table 1: Initial rate sweep, before the ARP fix (Section 5.2). The same code, cohort, and send pattern succeeded twice and failed once at 8.0 Gbit/s.

The same code, the same cohort, and the same send pattern succeeded twice and then failed at a rate previously confirmed safe. This ruled out a simple offered-rate threshold as the explanation and initially read as intermittent hardware or third-party-IP misbehavior — exactly the diagnosis we were reluctant to accept without direct evidence, because it is the diagnosis that most readily excuses not looking further.

**Root cause.** `ip neigh show` on the sending host showed the FPGA’s neighbor-table entry in **STALE** state after any idle interval between runs (editing a script, a `git commit`, or simply the wall-clock gap introduced by our own test orchestration). We confirmed this mattered operationally, not just cosmetically: five isolated `ping -c1` probes, spaced one second apart, lost roughly one in four; a single continuous run of forty pings at 0.3 s intervals lost none. This is consistent with Linux’s neighbor-discovery cache requiring reachability reconfirmation after its reachable-time window elapses, and with a UDP send issued while that reconfirmation is outstanding being more likely to be delayed or dropped than one issued into an already-warm entry — a plausible, though not exhaustively instrumented, mechanism, since we did not capture kernel neighbor-subsystem tracing to confirm the exact drop point.

We also found, in the course of diagnosing this, an unrelated orchestration bug that had been actively masking the ARP hypothesis: our test harness used `set -e` and started the FPGA-side

controller process in the background before the ARP-sensitive step; when that later step failed, the shell script exited without terminating the still-running, still-waiting controller process, which then held the device’s exclusive context open. The *next* invocation failed immediately with an unrelated XRT error (`failed to open cu context: Invalid argument`) that looked like a fresh, distinct failure rather than a downstream symptom of the first one. We mention this because it is a common shape of compounding failure in iterative hardware bring-up — a real bug’s symptom is overwritten by an orchestration bug’s symptom before the real bug is understood — and because the fix (a `trap`-based cleanup that terminates an armed-but-orphaned controller process on any subsequent failure) is generic and reusable beyond this project.

**Fix and validation.** We added a three-packet ICMP warm-up (`ping -c3, 0.2 s` spacing) immediately before every data burst, forcing bidirectional reachability confirmation (the echo reply confirms the sender’s neighbor entry; producing that reply requires the FPGA’s ARP responder to have the sender resolved as well) before UDP traffic begins. We validated this fix under conditions at least as adverse as those that triggered the original failures: four runs immediately following the fix, the last with the sending host’s neighbor-table entry for the FPGA *forcibly deleted* beforehand (`ip neigh del`) to guarantee a cold-start resolution rather than merely a stale one — all four passed bit-exact with zero data loss. We then ran a ten-run confirmation batch at 4,000,003 samples and 8 Gbit/s with no manual intervention between runs beyond the automated warm-up: **10/10 passed**, with zero recoveries needed, reported in full in Section 4.

### 5.3 What we can and cannot claim about throughput

We report 8 Gbit/s ( $\approx 62.5$  Msamples/s) as a **sustained, reproducible** network-path throughput, evidenced by ten consecutive clean runs with no failures and no recovery interventions. We do not claim this is the network path’s ceiling — 12 Gbit/s succeeded in a single run before the ARP-staleness confound was identified and controlled for, and the theoretical bounds are considerably higher (100 Gbit/s Ethernet line rate; 120.1 Gbit/s of kernel bus bandwidth at the implemented 234.6 MHz clock) — only that 8 Gbit/s is the rate at which we have direct, repeated, controlled evidence of reliability. Determining the true ceiling would require re-running the rate sweep of Section 5 with the ARP fix in place throughout, which we have not yet done; we flag this as the immediate next step rather than extrapolate it. For comparison, the previously-accepted DMA-attached variant of the same kernel sustains 511 Msamples/s ( $\approx 65.4$  Gbit/s of raw data) — the network-attached path, at its currently-confirmed rate, is roughly  $8\times$  slower than DMA. Whether that gap closes with a faster host-side sender (the confirmed ceiling was bound by single-threaded `sendto` syscall throughput, not by the fabric or the kernel) or persists as a structural cost of network attachment is open.

## 6 Discussion

The central methodological point of this work is not the specific throughput number, which we expect to improve, but the diagnostic path that got us from “the third-party network core is unreliable” to “the host’s ARP cache was stale.” Every piece of evidence available at the network layer — packet loss that did not correlate cleanly with offered rate, a device that stopped responding to trivial cohorts after a failure, recovery only via full bitstream reload — was individually consistent with a hardware or third-party-IP defect, and none of it, on its own, pointed at the host’s neighbor-discovery cache. What did point there was a narrow, falsifiable observation (isolated pings losing at a markedly higher rate than a continuous ping stream) obtained by testing the simplest possible mechanism first (ICMP, no FPGA logic involved)

once the failure pattern (works right after another run, fails after a gap) suggested *time since last traffic*, not *offered rate*, as the operative variable. We think this generalizes: when a network-attached accelerator’s failure signature correlates with wall-clock gaps between tests rather than with load, the host IP stack’s own caches are worth checking before the accelerator’s logic.

## 7 Threats to validity and future work

- The root-cause mechanism for ARP staleness causing UDP loss is supported by a strong behavioral correlation (isolated-ping loss vs. continuous-ping non-loss; failure history correlating with wall-clock gaps) and by the fix eliminating the failure across five subsequent test batches, but we did not directly instrument the kernel’s neighbor subsystem or capture the dropped frames at the point of loss, so an additional or interacting factor cannot be fully excluded.
- The confirmed-safe 8 Gbit/s figure reflects ten runs at that specific rate; it is not a swept characterization with the ARP fix applied throughout, and the true throughput ceiling of the network path — and whether it is bound by the host sender, the fabric, or the third-party network core — remains open.
- All hardware validation in this work used a single Alveo U250 on a shared cluster fabric; we have not tested a second card or a different network topology, and cannot yet distinguish card-and-fabric-specific behavior from properties of the VNx IP core in general.
- Latency (as opposed to throughput) of the network-attached path relative to the DMA-attached baseline is not measured in this work and is the more direct motivation for network attachment in a disaggregated pipeline; we defer it to a follow-up measurement now that a reliable transmission method exists.

## Reproducibility

All source — the ported kernel, HLS testbench, the two-process host controller and packet injector with an embedded golden model, two independent wire-format verifiers, and orchestration scripts including the ten-run confirmation batch — is available in `fpga/san-net/` in this repository, alongside a complete, unedited build and debugging log in `fpga/san-net/README.md` covering every failure and fix described in Section 5, timestamped as they occurred.